

FalconStrix Security Assessment

Cross-Site Scripting (XSS) Testing & Alert Resolution Review

Target application:	FalconStrix
Test date:	July 27, 2026
Tester:	Internal security testing
Scope of this report:	Alert description field (stored XSS test), Resolved Cases rendering, alert resolution workflow

Executive Summary

Testing focused on whether the alert **Description** field is vulnerable to stored Cross-Site Scripting (XSS). A test payload was submitted, stored in the backend without modification, and rendered back to the browser as HTML-escaped plain text rather than executing as script. This is the expected, safe behavior. A second test attempted to verify whether the same stored payload renders safely in a different view (the Resolved Cases page), since escaping is only effective if applied consistently everywhere the data is displayed. That second test could not be completed because of an unrelated functional bug in the alert-resolution workflow, which is documented separately below.

#	Item	Result	Severity
A	Stored XSS test - Live Alerts table (Description field)	Not vulnerable - output escaped	N/A
B	Storage-layer check (raw payload persistence)	Confirmed - escaping applied at render, not input	Informational
C	Stored XSS test - Resolved Cases page (same payload)	Untested - blocked by functional bug	Open
D	Alert resolution workflow (Solve action)	Functional bug - resolved alert not appearing in Resolved Cases	Functional (no security)

Note: Item C is an open item, not a confirmed vulnerability. Item D is a functional defect that blocked completion of item C's test — it is not itself an XSS finding.

Part A - Stored XSS Test: Live Alerts Table (Description Field)

Test payload

```
<script>alert('XSS-TEST')</script>
```

Method

The payload above was submitted as the Description value when creating a new alert (case FALSTRX), then the resulting alert was viewed in the Live Alerts table to see how the message rendered.

Result: **Not vulnerable**

The Description column displayed the payload as literal, visible text - the characters `<script>alert('XSS-TEST')</script>` as shown on screen - rather than executing it. No JavaScript popup fired from the payload itself. (A popup reading "MEDIUM alert created and sent to backend" did appear, but this is the application's standard confirmation message shown after creating any alert, and is unrelated to the payload.)

Mechanism

This behavior is consistent with HTML escaping (output encoding) applied at render time. Rather than inserting the raw message into the page's HTML, the frontend converts special characters (`<`, `>`, `"`, etc.) into their safe equivalents before display. As a result the browser never interprets the payload as a real `<script>` tag - it only ever sees text.

Part B - Storage-Layer Confirmation

The raw backend response (captured via Burp Suite) for the created alert showed the message field stored exactly as submitted, with no modification:

```
"message": "<script>alert('XSS-TEST')</script>"
```

This confirms the escaping observed in Part A happens at render time on the frontend, not by stripping or rejecting the payload on input. Storing data raw and escaping it consistently at every output point is a standard, acceptable pattern - **provided every place the data is displayed applies the same escaping**. That consistency is what Part C was intended to verify.

Part C - Stored XSS Test: Resolved Cases Page

Objective

Confirm whether the same stored payload renders safely (escaped) when displayed in a second, different view - the Resolved Cases page - which may use a different template or rendering path than the Live Alerts table tested in Part A.

Planned method

1. Resolve the FALSTRX alert containing the test payload.
2. Locate the same alert on the Resolved Cases page.
3. Inspect the DOM (not just the visual result) to check whether the payload appears as escaped text or as a live tag.
4. If the first payload did not execute, retest with an `<img src=x onerror=alert('XSS-TEST-2')>` payload, since onerror handlers can fire even when injected via unsafe HTML insertion methods that a plain `<script>` tag would not.

Result: **Not completed**

This test could not be carried out because the alert did not move to the Resolved Cases page after the Solve action was triggered (see Part D). **This item remains open** - the Resolved Cases rendering path has not yet been verified as safe, and should be retested once the resolution workflow is fixed.

Part D - Functional Defect: Alert Resolution Workflow

This is a functional bug, not a security finding, but it is documented here because it blocked Part C and affects confidence in the resolution workflow generally.

Observed behavior

Clicking Solve/Resolve on the test alert returned a success response, but the alert did not appear on the Resolved Cases page. Re-clicking Solve on the same alert returned:

```
"ok": true, "message": "Case already resolved"
```

This confirms the backend had already marked the alert as resolved on the first click - the write operation succeeded. The bug is therefore on the read/query side: either the Live Alerts view is not excluding resolved alerts, or the Resolved Cases view is filtering on a condition that incorrectly excludes this (and possibly other) resolved rows.

Secondary observation

The resolve response body was unusually large (approximately 122 KB) for what would typically be a small confirmation payload. This warrants a follow-up check of exactly what the response contains - for example, whether it is returning a full alerts list on every resolve call, whether that list is correctly scoped to the current account/severity, and whether the resolved alert is present in it with the correct status field (which would point to a frontend rendering/routing bug rather than a backend query bug).

Recommended next steps

1. Review the Resolved Cases page's query/filter logic against the alerts table's status field.
 2. Confirm the Live Alerts view explicitly excludes resolved alerts.
 3. Once resolution correctly moves alerts between views, re-run Part C to close out the Resolved Cases XSS test.
 4. Separately, verify the PDF report export path and the username field for the same class of unescaped-output issue, since escaping applied in one place does not guarantee it is applied everywhere.
-

This report reflects testing completed to date. Item C (Resolved Cases XSS rendering) remains open pending resolution of the Part D functional defect, and should not be reported as a closed or passed test until it has actually been verified.